ECE 786 Topics in Advanced Computer Architecture: Data Parallel Processors

Prof. Huiyang Zhou

Course Description (Syllabus)

- Energy-efficient high-performance computing
  - Processor core architecture
  - Memory systems
  - Programming models
- Focus:
  - Data-level parallelism, thread-level parallelism
  - GPU computing/GPGPU
  - Vector Processors
  - FPGAs
- Prerequisite
  - ***ECE563 and ECE/CSC506: Computer Architecture Concepts including pipelining, memory hierarchy, etc.***
  - C/C++ programming

Description (cont.)

- Textbook
  - No required textbooks, a few optional ones
  - Papers & Notes

- ***Tentative*** grading policy
  - +/- policy will be used
  - Programming assignments (CUDA/OpenCL programming assignments + simulator learning mini-projects) 24%
  - Midterm exam 25%
  - Final exam 25%
  - Homeworks 10%
  - Final project 16%

Outcomes/Objectives

- **Understand the key ideas behind the GPU compute architecture and programming model**
- **Explain the functionality of each component in a diagram of common GPGPU architecture.**
- **Write CUDA/OpenCL programs**
- **Translate a CUDA/OpenCL program into assembly and determine the execution result (i.e., machine states) based on the assembly code**
- **Show the cycle-by-cycle behavior of GPGPU pipelines for a sequence of assembly instructions**
- **Analyze the performance of CUDA/OpenCL programs running on GPUs (Performance model, Memory performance)**
- **Explain how an OpenCL program is synthesized onto FPGAs.**
- **Explain the functionality of each component in a diagram of common vector processor architecture.**
- **Review the literature on research topics focusing on data parallel processors**
- **Evaluate the pros and cons of two main data parallel processor architectures: GPGPU vs. Vector processors => GCN architecture**

Assignments

- Homeworks
  - Based on lecture material
  - Help to prepare you for exams
- Programming Assignments
  - Develop CUDA/OpenCL programs & optimize them
  - Mini projects to learn the GPGPUsim simulator infrastructure
- Course Projects
  - Simulator-oriented (Focusing on architecture)
    - Performance analysis
    - Architectural changes to improve performance, power efficiency, etc.
    - Can be team project (based on a research paper) (opt. 1)
    - Or individual project (assigned by the instructor) (opt. 2)
  - Application-orientated (focusing on code optimization) (opt. 3)
    - Select (or find your own) an important application
    - Develop CUDA/OpenCL code
    - Improve the performance/security/reliability (need insights)

Experiments

- CUDA/OpenCL programs
  - GPUs are widely available
    - Check your laptop/desktop
    - Departmental GPU cluster: hydra.ece.ncsu.edu
  - Emulation-based
    - OK for programming assignments
    - Not for course projects

- GPGPU simulators
  - Several open-source simulators available
    - AccelSim (UBC & Purdue)
    - Ocelot and SST-MacSim Simulator (Gatech)
    - MGPUSim (W&M, Northeastern U)
    - Gem5+GCN (AMD)
    - Vortex (http://vortex.cc.gatech.edu/) (More on HW)

Who am I

- Professor at ECE Dept.

- My research area: computer architecture, back-end compiler
  - High Performance, Power/Energy Efficient, Fault Tolerant Microarchitectures, Multi-core/many-core architectures (e.g., GPGPU), Architectural support for software debugging, Architectural support for information security, Quantum computing

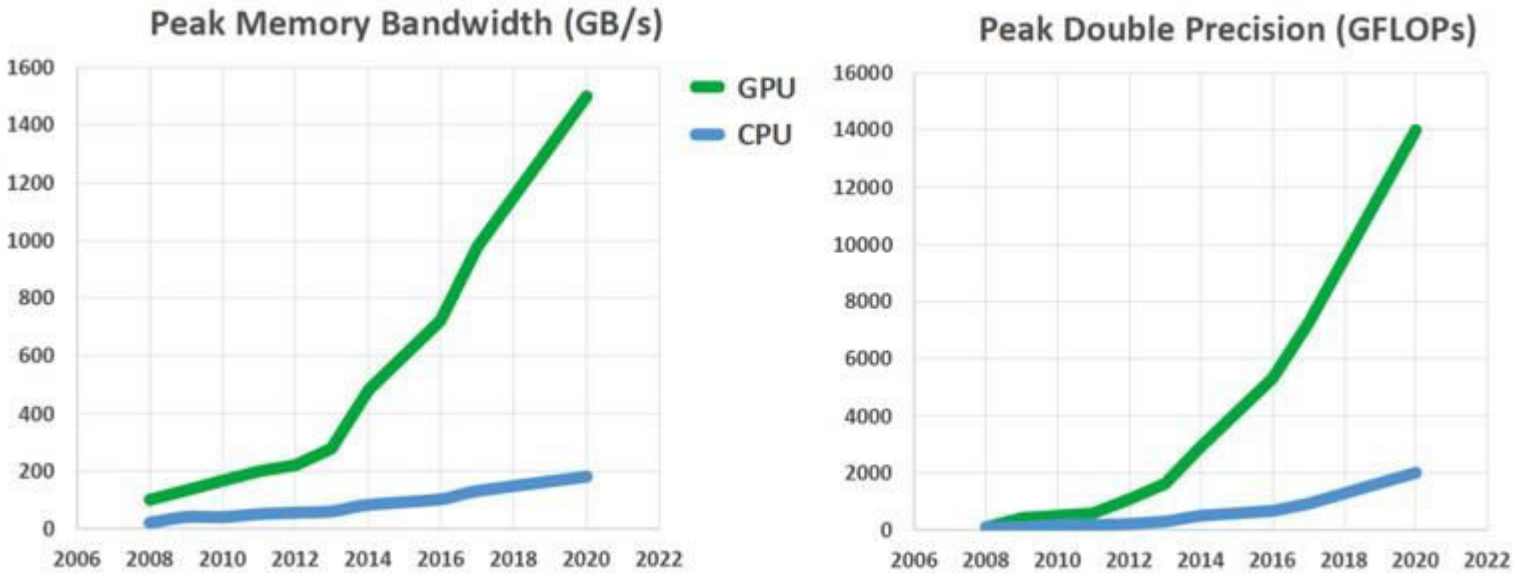
Acknowledgement

- Some material including lecture notes are based on the lecture notes of the following courses:

- Programming Massively Parallel Processors (UIUC)

- Computer Architecture: A Quantitative Approach (7th ed.) Chapter 4.
- I wrote a book chapter on SIMT architecture in **Fundamentals of Parallel Multicore Architecture, Chapman & Hall/CRC Computational Science, 1st Ed**

GPU Evolution

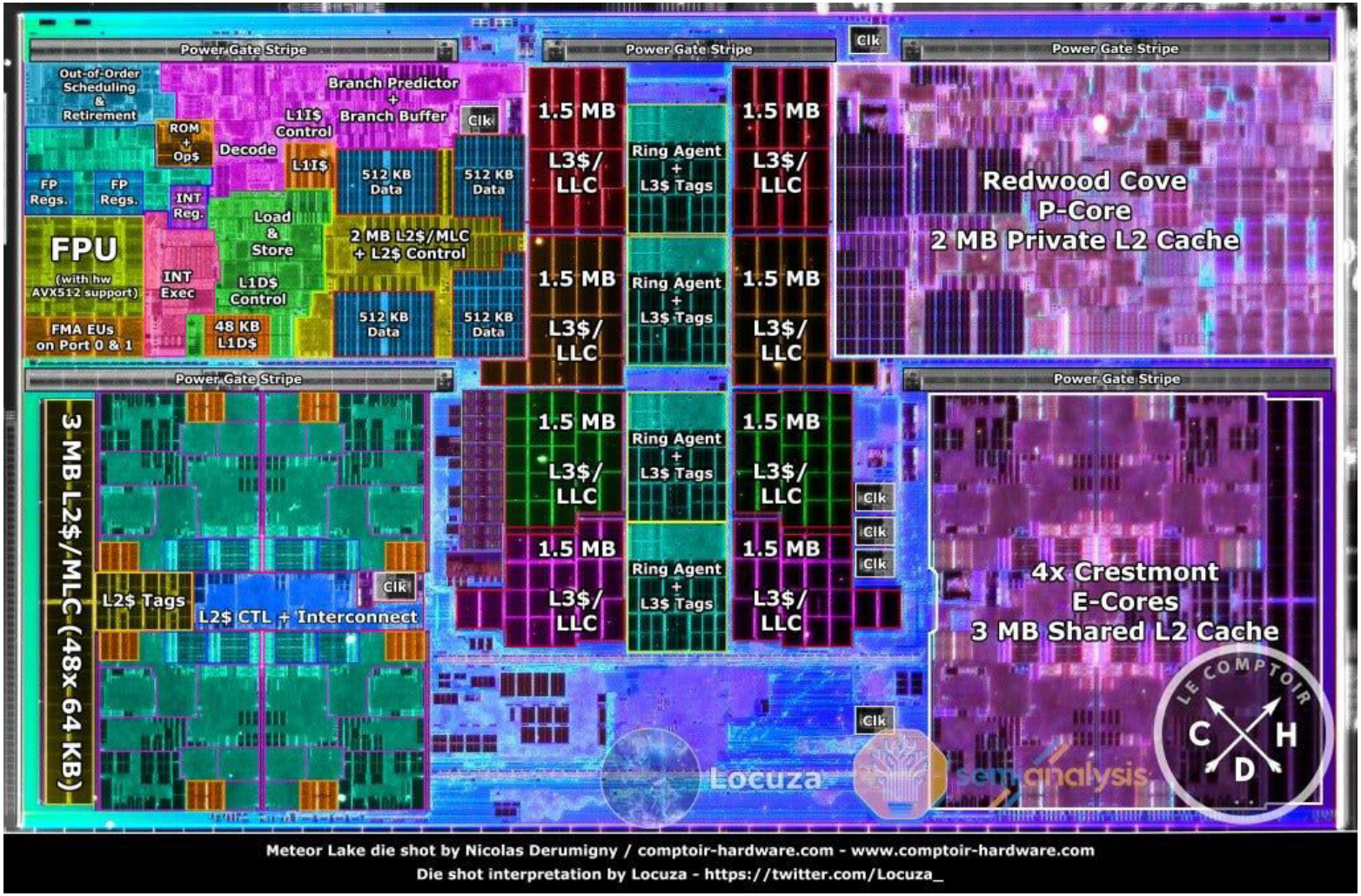


https://www.nextplatform.com/2019/07/10/a-decade-of-accelerated-computing-augurs-well-for-gpus/

Die photo (CPU)

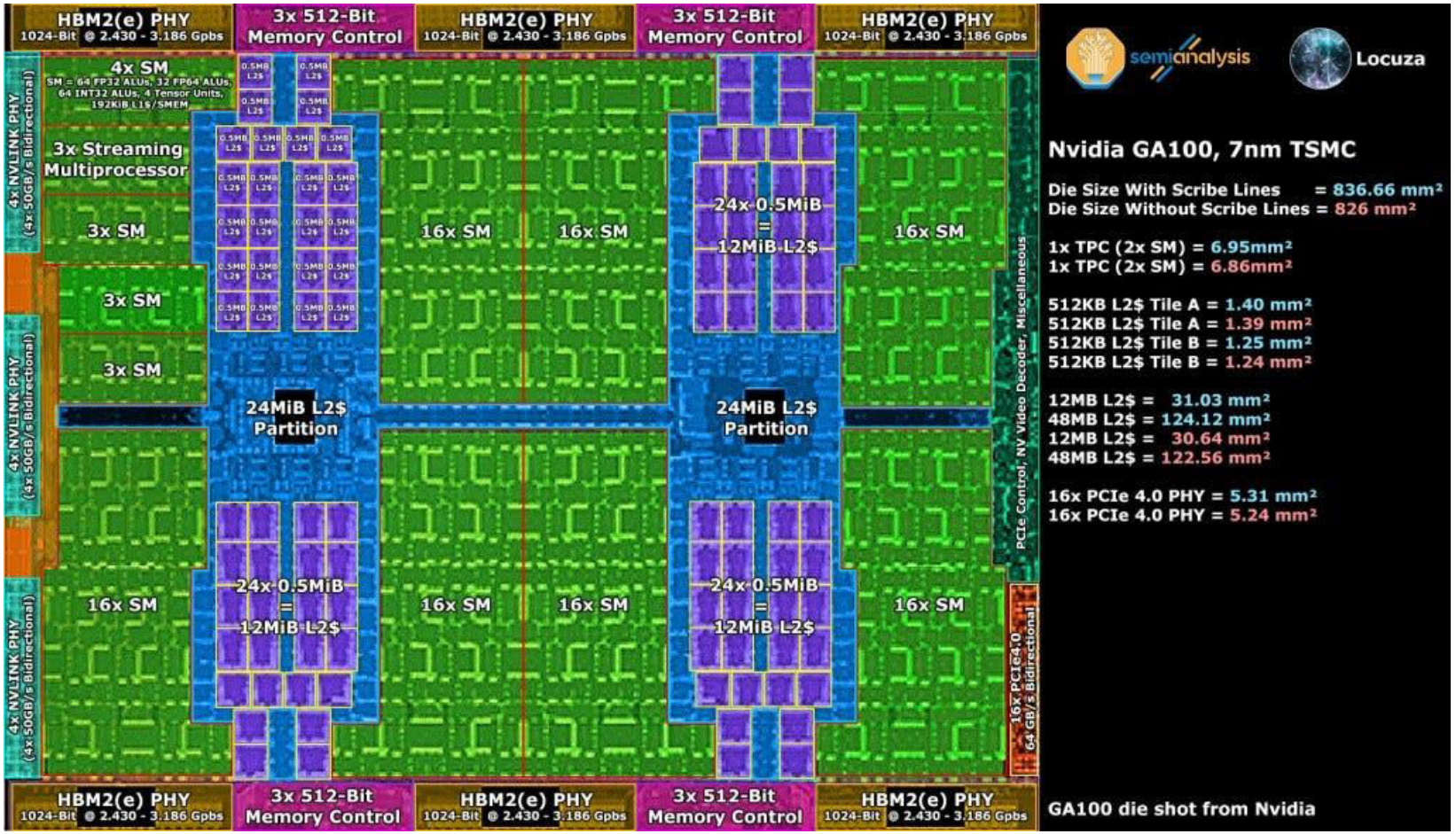


Die photo (CPU)



https://semianalysis.com/2022/05/26/meteor-lake-die-shot-and-architecture/

Die photo (GPU)



GPU Evolution: Motivation

- Technology Push: power wall & memory wall
  - Solution: Data-level parallelism (thread-level parallelism)

- Application Pull: Processing high volumes of data
  - Implication 1: Move data fast => High memory bandwidth (GB/s => TB/s)
  - Implication 2: Perform computation fast => High computational throughput (GFLOPS => TFLOPS)
  - Many applications are more throughput sensitive than latency sensitive
    - E.g., Graphics. FPS => 1920x1080 pixels/s instead of the delay of each pixel
  - Solution: More on-chip compute resources than memory, Scalable architecture

Computer Science at a Crossroads (D. Patterson)

- Old: Uniprocessor performance 2X / 1.5 yrs
- New: Power Wall + ILP Wall + Memory Wall = Brick Wall
  - Uniprocessor performance now 2X / 5(?) yrs

  ⇒ Sea change in chip design: multiple "cores"
  (2X processors per chip / ~ 2 years)
    - More simpler processors are more power efficient
- The Free (performance) Lunch is over: A Fundamental Turn Toward Concurrency in Software
  - The biggest sea change in software development since the OO revolution is knocking at the door, and its name is Concurrency (by Herb Sutter)

Problems with Sea Change

- Algorithms, Programming Languages, Compilers, Operating Systems, Architectures, Libraries, … not ready to supply Thread Level Parallelism or Data Level Parallelism for 1000 CPUs / chip,
- Architectures not ready for 1000 CPUs / chip
  - Unlike Instruction Level Parallelism, cannot be solved by just by computer architects and compiler writers alone, but also cannot be solved *without* participation of computer architects
  - Modern GPUs run thousands of threads / chip
- Shifts from Instruction Level Parallelism to Thread Level Parallelism / Data Level Parallelism
  - GPGPU/GPU computing is one such example

- Domain specific architectures

Power/Energy Consumption

- Energy per operation (where are my precious Joules?)
  - Based on a 32-bit RISC processor (0.13um, 1.2V) [W. Dally, et. al., 2004]

Operation	Energy
32-bit arithmetic operation	5pJ
32 bit register read	10pJ
32-bit 8kB RAM read	50pJ
32-bit traverse 10mm wire	100 pJ
Execute one instruction	500 pJ

On a more recent technology (45nm)

Integer	
Add	
8 bit	0.03pJ
32 bit	0.1pJ
Mult	
8 bit	0.2pJ
32 bit	3.1pJ

FP	
FAdd	
16 bit	0.4pJ
32 bit	0.9pJ
FMult	
16 bit	1.1pJ
32 bit	3.7pJ

Memory	
Cache	(64bit)
8KB	10pJ
32KB	20pJ
1MB	100pJ
DRAM	1.3-2.6nJ

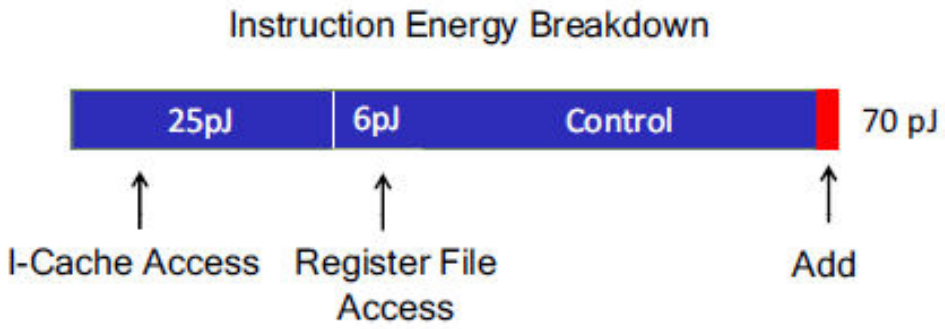


Figure 1.1.9: Rough energy costs for various operations in 45nm 0.9V.

From "Computing's energy problem" by M. Horowitz (ISSCC 2014)

Processor Level Energy Distribution

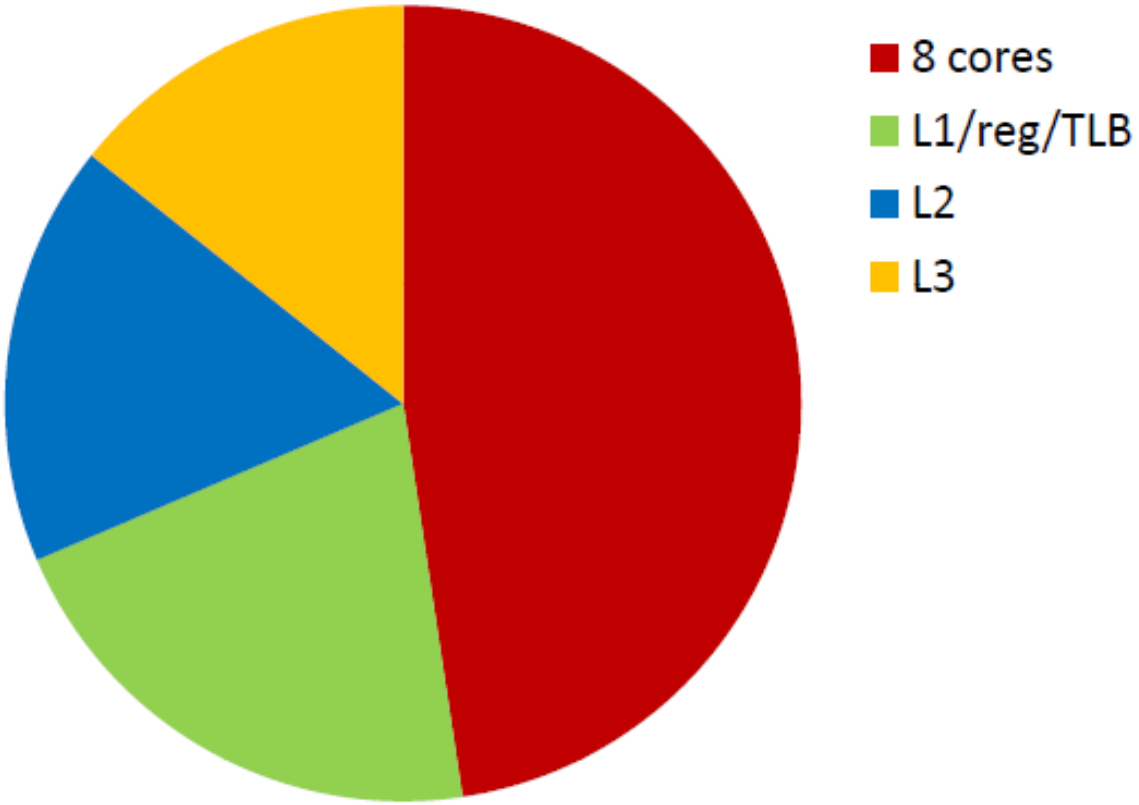


Figure 1.1.7: Power breakdown of an 8 core server chip.

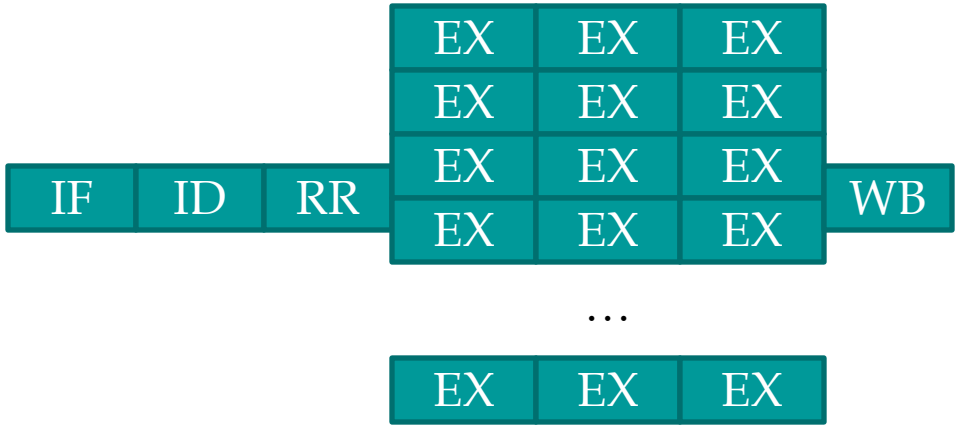
From “Computing’s energy problem” by M. Horowitz (ISSCC 2014)

Data Parallel Processors

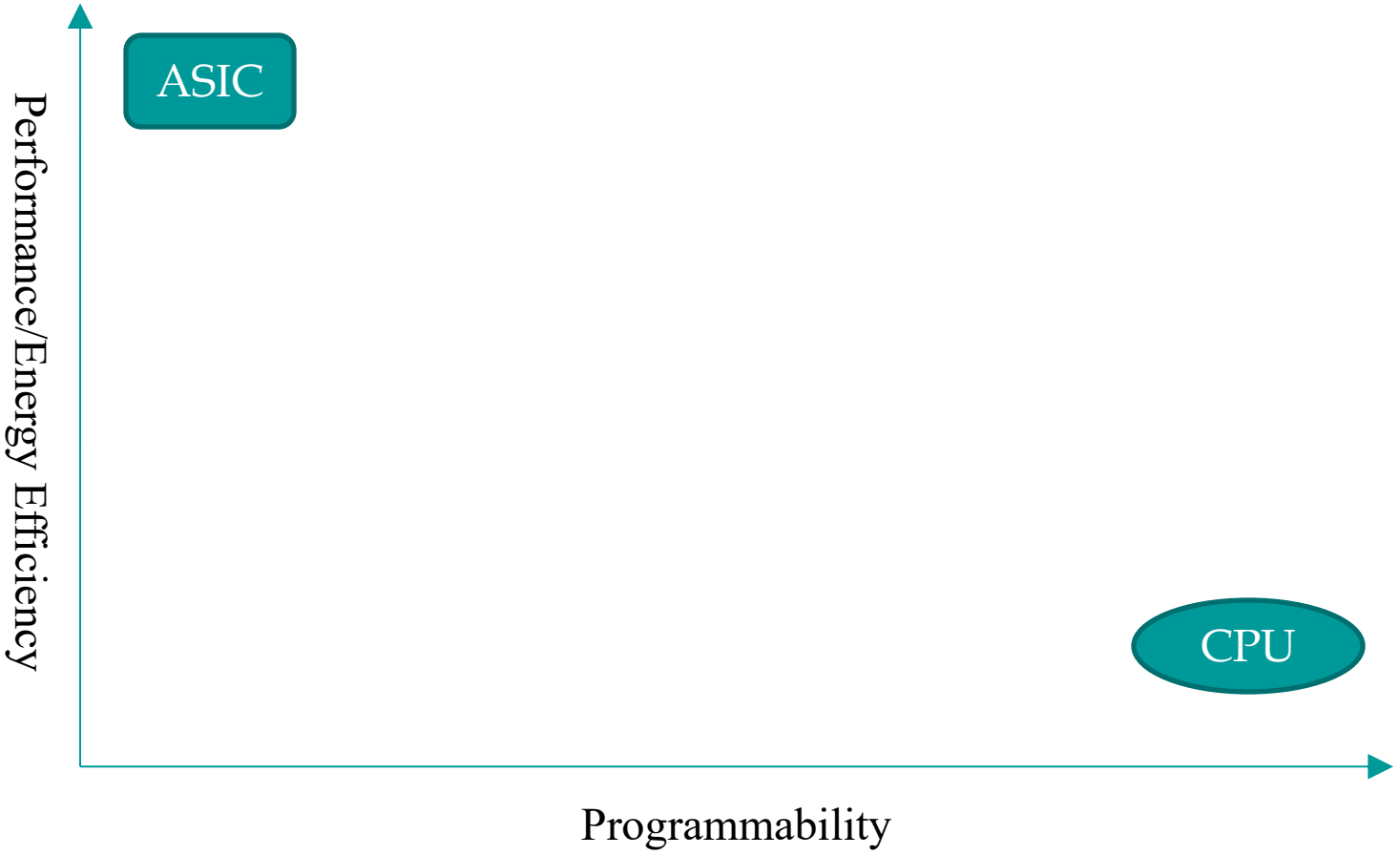
- Data Level Parallelism (DLP)
  - Same operations on many data items
- Energy-efficiency: amortize the instruction fetch/decode/control for multiple data items
  - Same Instruction Multiple Data (SIMD)
- Key data parallel processing paradigms
  - Vector Processors
  - SIMD Instruction Extensions
  - GPU/GPGPU

Basic Data Parallel Processor Architecture

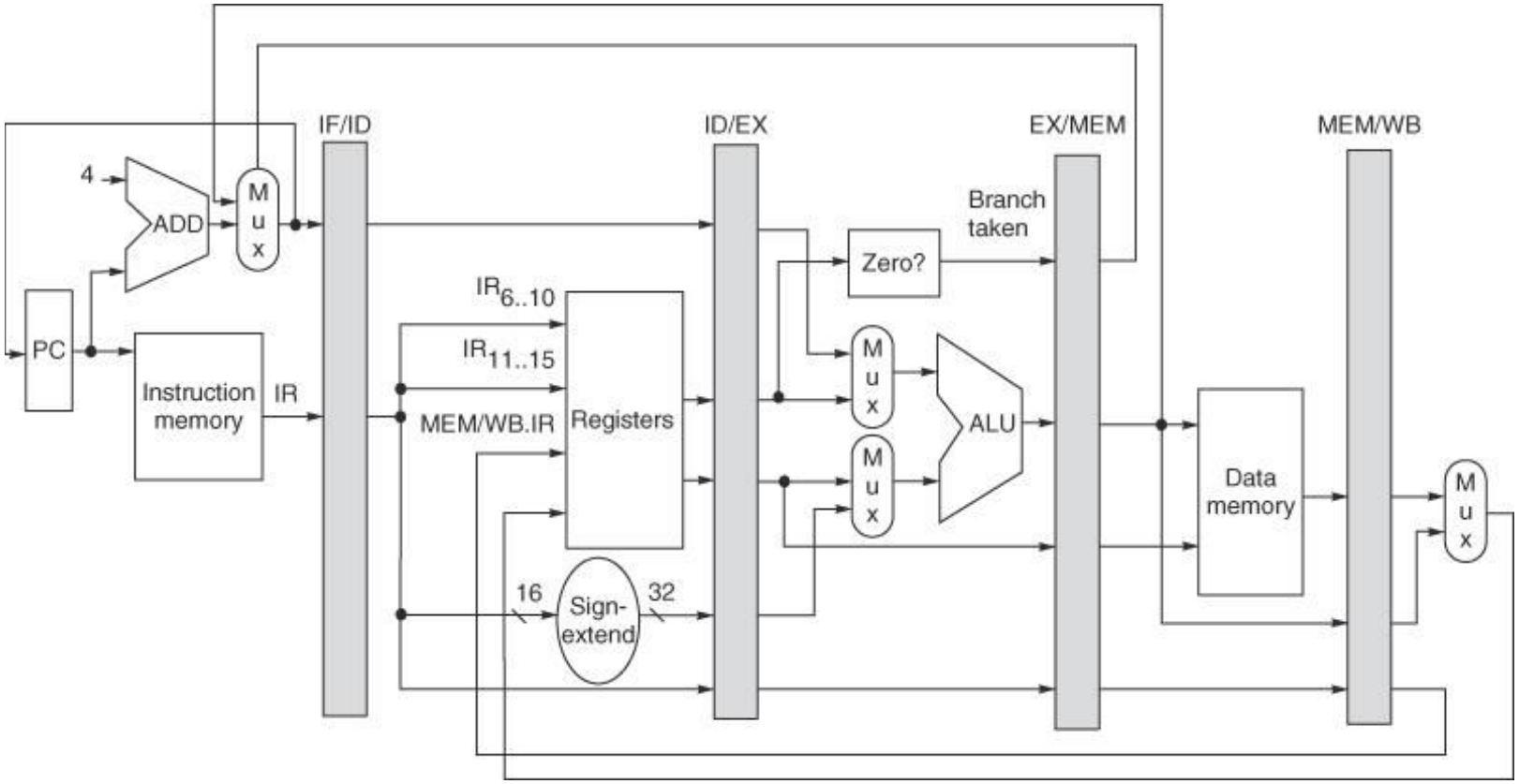
- Performance and Energy Efficiency come from data-level parallelism
  - Lead to relatively simple pipeline architecture
  - In-order pipeline execution



Performance/Energy Efficiency vs. Programmability



Review of In-order Scalar Pipelines



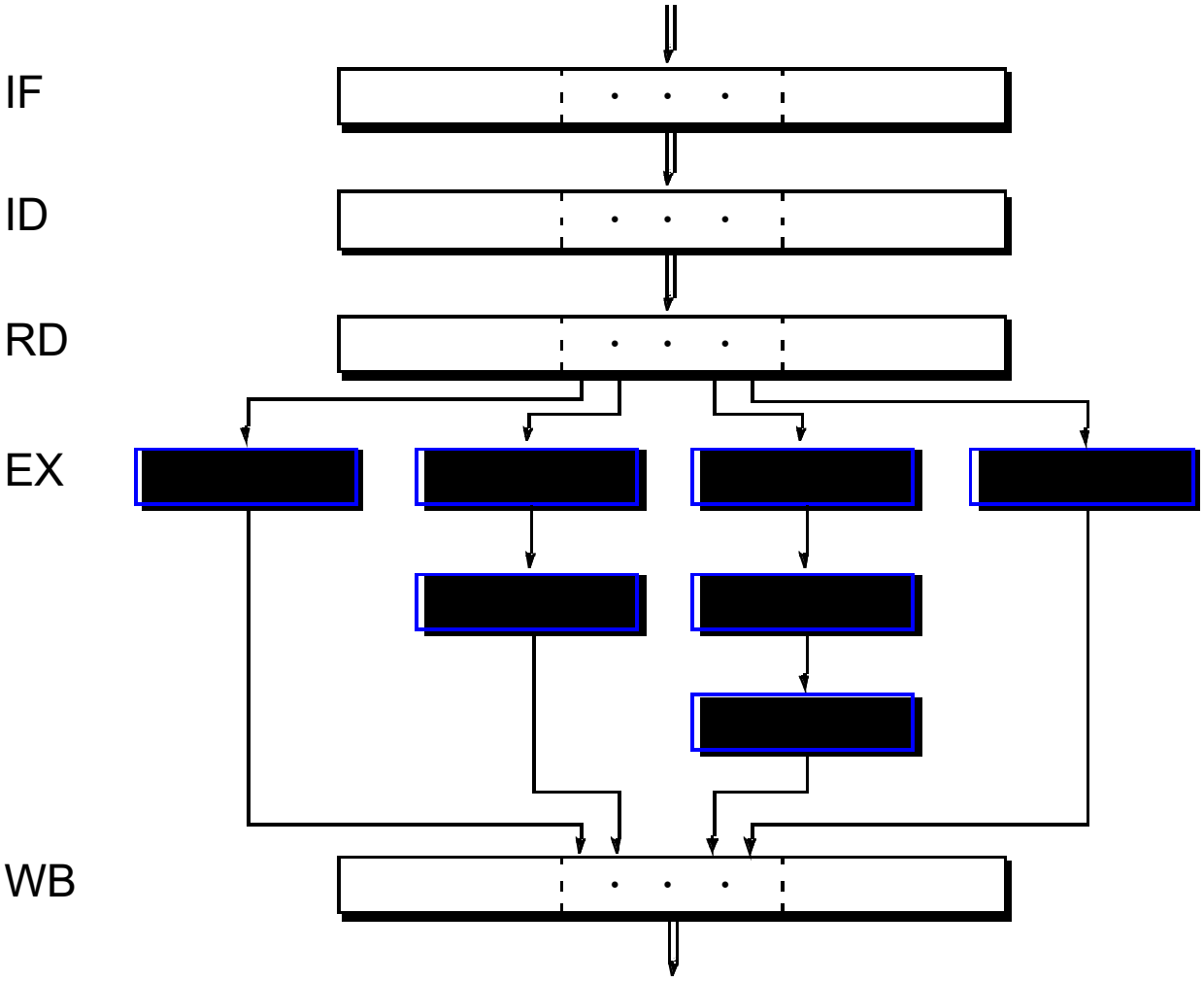
Pipelined Execution

Insn 1	IF	ID	EX	MEM	WB		
Insn 2		IF	ID	EX	MEM	WB	
Insn 3			IF	ID	EX	MEM	WB
Insn 4				IF	ID	EX	MEM
Insn 5					IF	ID	EX
…						IF	ID

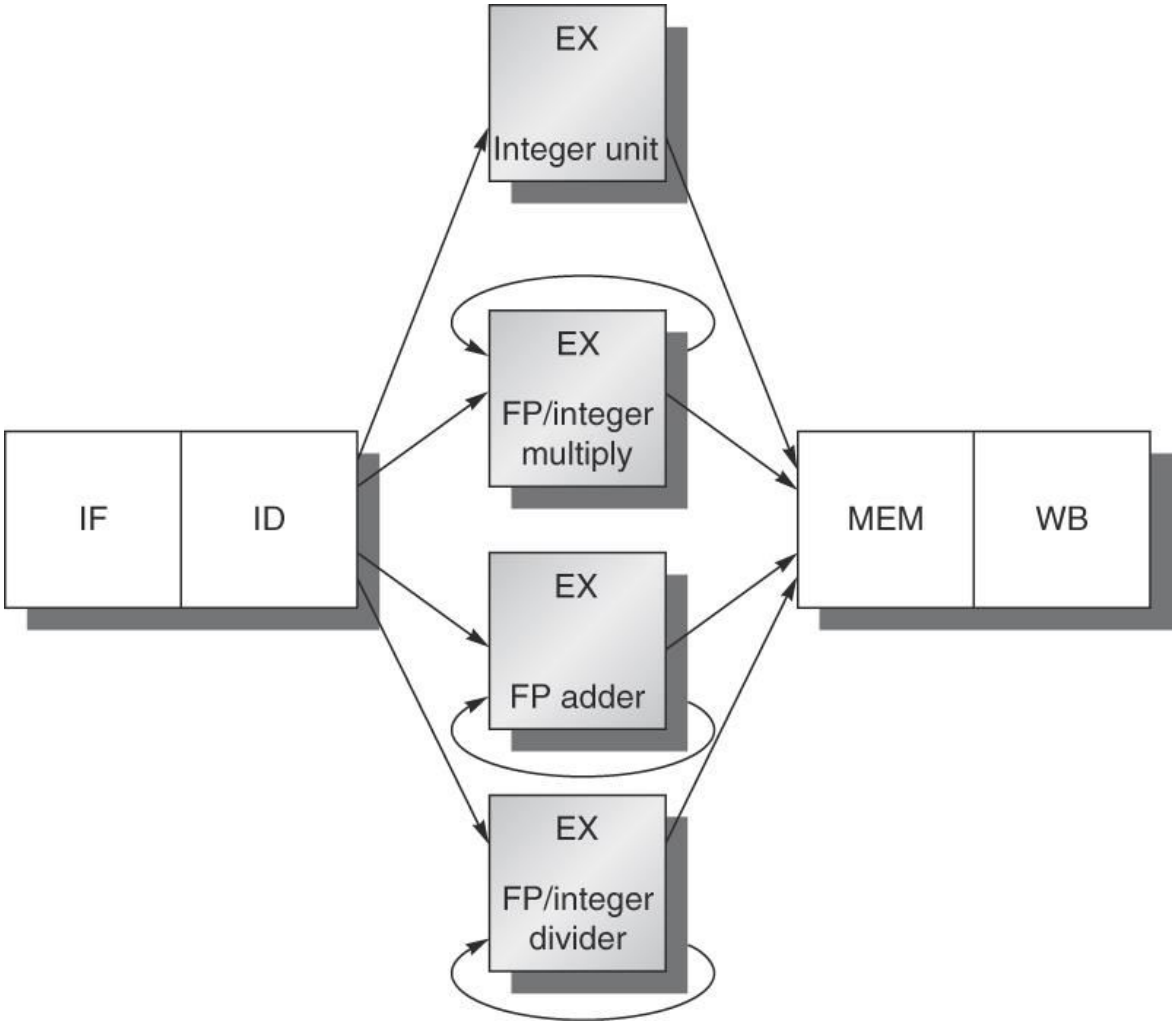
Pipeline Hazards

- Structural hazards
- Data hazards
  - RAW
  - WAR
  - WAW
- Control Hazard

Diversified Pipelines



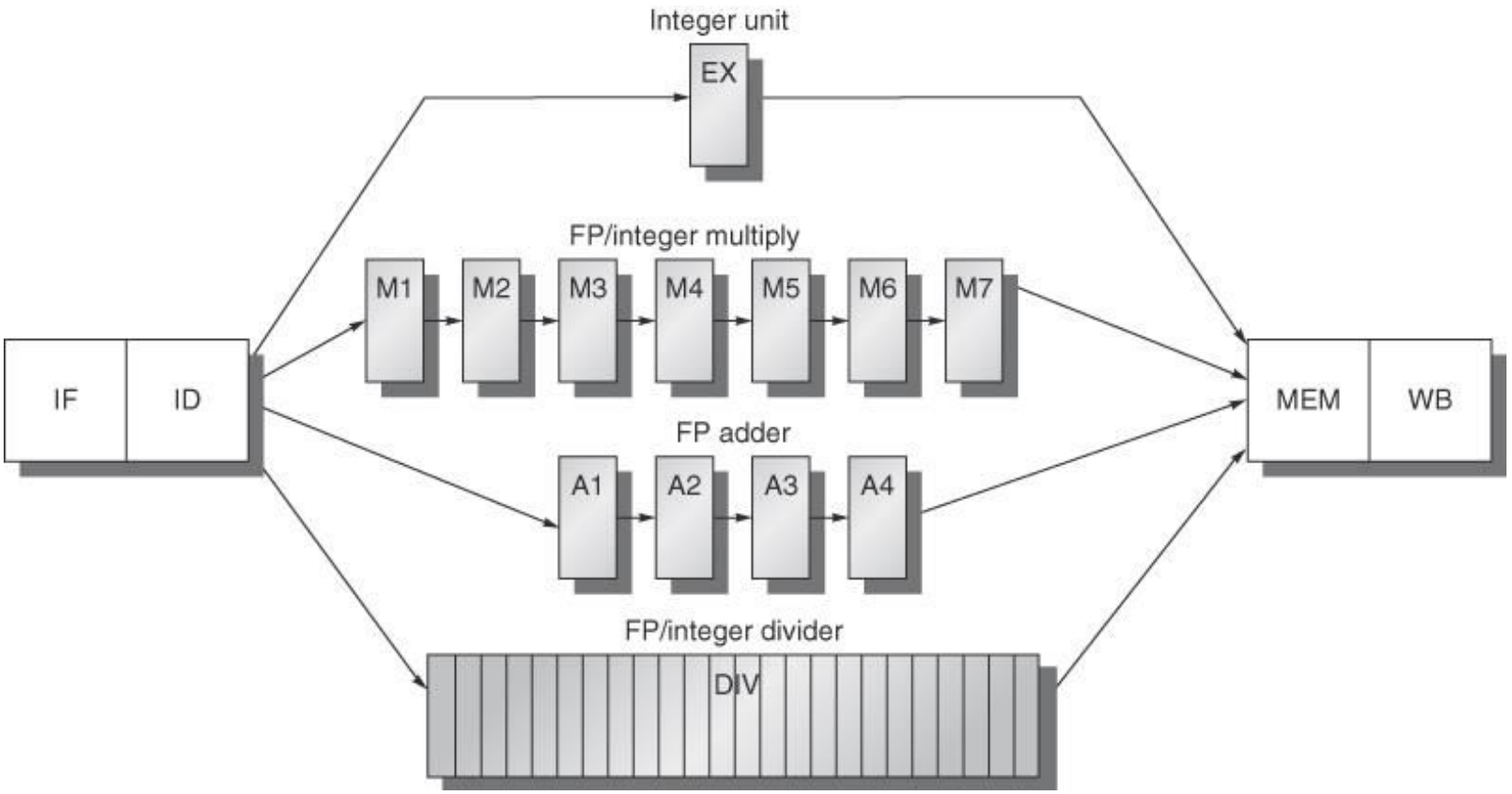
MIPS Pipeline with (un-pipelined) FP units



Problems with (un-pipelined) FP units

- Hazards
  - Which type

MIPS Pipeline with pipelined FP units (cont.)



Pipelined Function Units: Latency and Initialization Interval

Functional Units	Latency	Initiation Interval
Integer ALU	0	1
Loads	1	1
FP Add	3	1
FP Multiply	6	1
FP Divide (unpipelined)	24	25

Pipeline Data Hazards

- WAR
  - ADD.D F1, F2, F3 // F1=F2+F3
  - MUL.D F2, F5, F6 // F2 = F5 * F6
- WAW
  - MUL.D F2, F5, F6
  - …
  - ADD. D F2, F5, F6
- RAW (with data bypassing/forwarding)
  - L.D F4, 0(R2)
  - MUL.D. F0, F4, F6
  - ADD F2, F0, F8
  - S.D F2, 0(R2)

Pipeline Structural Hazards

MUL.D F0, F4, F6

ADD R1, R2, R3

ADD R10, R11, R12

ADD.D F2, F4, F6

ADD R5, R6, R7

ADD R8, R5, R12

L.D F2, 0(R2)

How to handle pipeline hazards

- Data Hazards
  - RAW
    - Bypassing and associated logic to detect whether the producer and consumer operands match (Hardware)
    - Bypassing and instruction encoding tells where to retrieve the source operands, i.e., the instruction encodes the multiplexer selection bits (Software)
    - May need to insert stall cycles for complex instructions (HW vs. SW)
  - WAW
    - Stall cycles depends on operand detection logic (HW)
    - Stall cycles determined by compiler (SW)
- Structural hazards

In-order Completion vs. Out-of-Order Completion

- DIV.D F0, F2, F4
- ADD.D F10, F10, F8
- SUB.D F12, F12, F14

Variable latency due to Loads

- Blocking (stall on miss)

Load r1, r2, #0	IF	ID	EX	MEM	s	s	s	…	s	WB	
Add r3, r4, r5		IF	ID	EX	s	s	s	…	s	MEM	
Load r6, r7, #8			IF	ID	s	s	s	…	s	EX	
Add r4, r1, #1				IF	s	s	s	…	s	ID	

- Non-blocking (stall on use or hit under miss)

Load r1, r2, #0	IF	ID	EX	MEM	s	s	s	…	s	WB	
Add r3, r4, r5		IF	ID	EX	M	WB					
Load r6, r7, #8			IF	ID	E	M	WB				
Add r4, r1, #1				IF	ID	s	s	…	s	EX	

There is hazard detection logic to ensure that when the miss is repaired, no contention on the RF write port (i.e., no two WB in the same cycle)